

渐近分析的记号

在下面的讨论中，对所有 n ， $f(n) \geq 0$ ， $g(n) \geq 0$ 。

(1) 渐近上界记号 O

$O(g(n)) = \{f(n) \mid \text{存在正常数 } c \text{ 和 } n_0 \text{ 使得对所有 } n \geq n_0 \text{ 有: } 0 \leq f(n) \leq cg(n)\}$

(2) 渐近下界记号 Ω

$\Omega(g(n)) = \{f(n) \mid \text{存在正常数 } c \text{ 和 } n_0 \text{ 使得对所有 } n \geq n_0 \text{ 有: } 0 \leq cg(n) \leq f(n)\}$

渐近分析的记号

(3) 非紧上界记号 o

$o(g(n)) = \{f(n) \mid \text{对于任何正常数 } c > 0, \text{ 存在正数和 } n_0 > 0 \text{ 使得对所有 } n \geq n_0 \text{ 有: } 0 \leq f(n) < cg(n)\}$

等价于 $f(n) / g(n) \rightarrow 0$, as $n \rightarrow \infty$ 。

(4) 非紧下界记号 ω

$\omega(g(n)) = \{f(n) \mid \text{对于任何正常数 } c > 0, \text{ 存在正数和 } n_0 > 0 \text{ 使得对所有 } n \geq n_0 \text{ 有: } 0 \leq cg(n) < f(n)\}$

等价于 $f(n) / g(n) \rightarrow \infty$, as $n \rightarrow \infty$ 。

$$f(n) \in \omega(g(n)) \Leftrightarrow g(n) \in o(f(n))$$

渐近分析的记号



(5) 紧渐近界记号 Θ

$\Theta(g(n)) = \{f(n) \mid \text{存在正常数 } c_1, c_2 \text{ 和 } n_0 \text{ 使得对所有 } n \geq n_0$
有: $c_1 g(n) \leq f(n) \leq c_2 g(n)\}$

定理1: $\Theta(g(n)) = O(g(n)) \cap \Omega(g(n))$

第一章:

代入

求证 $T(n) = 2T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + n$ 的渐近阶 $O(n \log n)$ 。

- 推测 $T(n) = O(n \log n)$ ，即推测存在正的常数 C 和自然数 n_0 ，使得当 $n \geq n_0$ 时有

$$T(n) \leq Cn \log n \quad (1-1)$$

- 假设假设当 $2^{k-1}n_0 \leq n \leq 2^k n_0$ ， $k \geq 1$ 时，(1-1) 成立。
- 那么，当 $2^k n_0 \leq n \leq 2^{k+1} n_0$ 时，有：

$$\begin{aligned} T(n) &= 2T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + n \\ &\leq 2C\left\lfloor \frac{n}{2} \right\rfloor \cdot \log\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + n \\ &< 2C \cdot \frac{n}{2} \cdot \log \frac{n}{2} + n \\ &= Cn \log n - Cn + n \\ &= Cn \log n - (C-1)n \\ &\leq Cn \log n \end{aligned}$$

迭代

求证 $T(n) = 3T\left(\frac{n}{4}\right) + n$ 的渐近阶为 $O(n)$

$$\begin{aligned} T(n) &\leq n + 3(n/4 + 3(n/4^2 + 3(n/4^3 + \dots + 3(n/4^i + 3T(0)) \dots))) \\ &= n + \frac{3}{4}n + \frac{3^2}{4^2}n + \frac{3^3}{4^3}n + \dots + \frac{3^i}{4^i}n + 3^{i+1} \cdot T(0) \\ &< 4n + 3^{i+1} \cdot T(0) \end{aligned}$$

知 $i \leq \log_4 n$ ，从而

$$3^{i+1} \leq 3^{\log_4 n + 1} = 3^{\log_3 n \log_4 3 + 1} = 3n^{\log_4 3},$$

代人

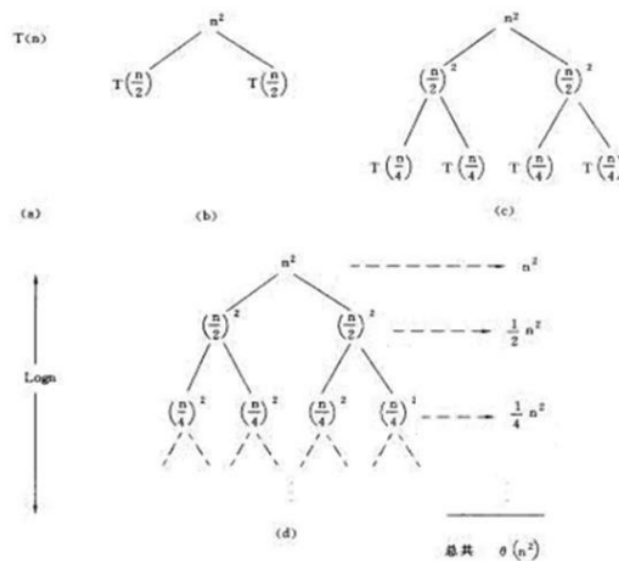
$$T(n) < 4n + 3n^{\log_4 3} \cdot T(0)$$

$$S_n = \frac{a_1(1-q^n)}{1-q} \quad (q \neq 1)$$

等比数列求和公式

求证 $T(n) = 2T\left(\frac{n}{2}\right) + n^2$ 的渐近阶为 $O(n^2)$

表格化的办法：先按横向求出每层结点的值之和，并记录在各相应层右端顶格处，然后从根到叶逐层地将顶格处的结果加起来便是我们要求的结果。照此，我们得到解的渐近阶为 $\theta(n^2)$ 。



套用公式

大小为 n 的原问题分成若干个大小为 n/b 的子问题，其中 a 个子问题需要求解，而 cn^k 是合并各个子问题的解需要的工作量。

$$T(n) = \begin{cases} c & n = 1 \\ aT(n/b) + cn^k & n > 1 \end{cases}$$

$$T(n) = \begin{cases} O(n^{\log_b a}) & a > b^k \\ O(n^k \log_b n) & a = b^k \\ O(n^k) & a < b^k \end{cases}$$

差分方程

$$\begin{cases} a_{n+2} = a_n + a_{n+1} & n \geq 0 \\ a_0 = 1, a_1 = 1 \end{cases}$$

该线性递归关系的特征方程 $x^2 = x + 1$

解得其根为
$$x = \frac{1 \pm \sqrt{5}}{2}$$

从而
$$a_n = c_1 \cdot \left(\frac{1+\sqrt{5}}{2}\right)^n + c_2 \cdot \left(\frac{1-\sqrt{5}}{2}\right)^n$$

将边界条件代入，得

$$\begin{cases} c_1 = \frac{5+\sqrt{5}}{10} \\ c_2 = \frac{5-\sqrt{5}}{10} \end{cases}$$

代入并化简后即得

$$a_n = \frac{1}{\sqrt{5}} \cdot \left(\frac{1+\sqrt{5}}{2}\right)^{n+1} - \frac{1}{\sqrt{5}} \cdot \left(\frac{1-\sqrt{5}}{2}\right)^{n+1}$$

求解常系数线性递归关系

$$\begin{cases} f(n) = 8f(n-1) + 9 \cdot 10^{n-2} \\ f(1) = 8 \end{cases}$$

方程的特征根为8，对应的通解为 $c \cdot 8^n$

设 $p \cdot 10^{n-2}$ 为一个特解，将它代入到递归关系中，得

$$p \cdot 10^{n-2} = 8p \cdot 10^{n-3} + 9 \cdot 10^{n-2}$$

解得 $p=45$ 。故非齐次递关系的通解为

$$f(n) = c \cdot 8^n + 45 \cdot 10^{n-2}$$

利用边界条件 $f(1)=8$ 求得 $c = \frac{7}{16}$

得 $f(n) = \frac{7}{16} \cdot 8^n + 45 \cdot 10^{n-2} = 7 \cdot 8^{n-2} + 45 \cdot 10^{n-2}$

母函数

例1 考虑线性变系数二阶齐次递归方程

$$(n-1)T(n)=(n-2)T(n-1)+2T(n-2), \quad n \geq 2$$

和初始条件 $T(0)=0, T(1)=1$ 。根据初始条件，可计算 $T(2)=0, T(3)=T(1)=1$ 。

设 $\{T(n)\}$ 的母函数为：

$$A(x) = \sum_{n=0}^{\infty} T(n)x^n$$

由于 $T(0)=T(2)=0, T(1)=1$ ，有：

$$\begin{aligned} A(x) &= x + T(3)x^3 + T(4)x^4 + \dots + T(n)x^n + \dots \\ &= x(1 + T(3)x^2 + T(4)x^3 + \dots + T(n)x^{n-1} + \dots) \end{aligned}$$

令 $B(x) = A(x)/x$ ，即：

$$B(x) = 1 + T(3)x^2 + T(4)x^3 + \dots + T(n)x^{n-1} + \dots$$

$$B(x) = 1 + T(3)x^2 + T(4)x^3 + \dots + T(n)x^{n-1} + \dots$$

那么：

$$B'(x) = 2T(3)x + 3T(4)x^2 + \dots + (n-1)T(n)x^{n-2} + \dots$$

$$xB'(x) = 2T(3)x^2 + 3T(4)x^3 + \dots + (n-1)T(n)x^{n-1} + \dots$$

$$B'(x) - xB'(x) = 2T(3)x + (3T(4) - 2T(3))x^2 + (4T(5) - 3T(4))x^3 + \dots + ((n-1)T(n) - (n-2)T(n-1))x^{n-2} + \dots$$

代入 $T(3)=1$ ，得

$$\begin{aligned} B'(x)(1-x) &= 2T(1)x + 2T(2)x^2 + 2T(3)x^3 + \dots + 2T(n-2)x^{n-2} + \dots \\ &= 2x(1 + T(3)x^2 + T(4)x^3 + \dots + T(n)x^{n-1} + \dots) \\ &= 2xB(x) \end{aligned}$$

即

$$\frac{B'(x)}{B(x)} = \frac{2x}{1-x} = -2 + \frac{2}{1-x}$$

第二章:(1实现原理,2代码,3证明复杂度,4画图展示过程)

二分搜索

//分治法：二分搜索

```
public static int binarySearch(int [] a, int x, int n)
{
    // 在 a[0] <= a[1] <= ... <= a[n-1] 中搜索 x
    // 找到x时返回其在数组中的位置，否则返回-1
    int left = 0; int right = n - 1;
    while (left <= right)
    {
        int middle = (left + right)/2;
        if (x == a[middle]) return middle;
        if (x > a[middle]) left = middle + 1; //在右半部查找
        else right = middle - 1;           //在左半部查找
    }
    return -1; // 未找到x
}
```

算法复杂度分析：

每执行一次算法的while循环，待搜索数组的大小减少一半。因此，在最坏情况下，while循环被执行了 $O(\log n)$ 次。循环体内运算需要 $O(1)$ 时间，因此整个算法在最坏情况下的计算时间复杂度为 $O(\log n)$ 。

归并排序

//合并算法

```
void Merge( Type c[], Type d[], int l, int m, int r)
{
    int i = l, j = m+1, k = 1;
    while(( i <= m ) && ( j <= r ))
    {
        if ( c[i] <= c[j] ) d[k++] = c[i++];
        else d[k++] = c[j++];
    }
    if ( i > m )
    {
        for (int q = j; q <= r; q++)
            d[k++] = c[q];
    }
    else
        for ( int q = i; q <= m; q++ )
            d[k++] = c[q];
}
```

//合并排序算法

```
void MergeSort ( Type a[], int n )
{
    Type *b = new Type [n];
    int s = 1;
    while ( s < n )
    {
        MergePass( a, b, s, n);
        s += s;
        MergePass( b, a, s, n);
        s += s;
    }
}
```

• 算法分析

- 最坏情况：归并排序是一个递归算法，所以很容易得到算法计算量的递推公式

$$W(n) = W(\lfloor n/2 \rfloor) + W(\lceil n/2 \rceil)$$
$$W(1) = 0$$

所以算法最坏情况的复杂度为 $\theta(n \log n)$

- 算法需要 $\theta(n)$ 的辅助空间

快速排序

(2) (7分) 快速排序过程如下

【答案1：以最左端元素 a[p] 为基准/划分元素，按照排序结果为非递减序列，划分、排序过程与书、讲义内容保持一致】

```
void QuickSort (Type a[], int p, int r) (2分)
{
    if (p < r) {
        int q = randomizedPartition(a, p, r);
        QuickSort (a, p, q-1); //对左半段排序
        QuickSort (a, q+1, r); //对右半段排序
    }
}
```

对 n 个元素组成的数组 $a[0:n-1]$ 进行排序，调用 QuickSort ($a, 0, n-1$)

以确定的基准元素 $a[p]$ ，对子数组 $a[p:r]$ 进行划分：

int Partition (Type a[], int p, int r) (3 分)

```
{
    int i = p, j = r + 1;
    Type x = a[p]; /*第 1 个元素
    // 将 < x 的元素交换到左边区域
    // 将 > x 的元素交换到右边区域
    while (true) {
        while (a[++i] < x);
        while (a[--j] > x);
        if (i >= j) break;
        Swap(a[i], a[j]);
    }
    a[p] = a[j];
    a[j] = x;
    return j;
}
```

int RandomizedPartition (Type a[], int p, int r) (2 分)

```
{
    int i = Random(p, r); //随机选 1 个 a[j]
    Swap(a[i], a[p]);      //将 a[i] 交换到 a[] 的第 1 个位置 a[p]
    return Partition (a, p, r);
}
```

【答案 2：随机选择算法挑选的划分基准元素交换到最左端 $a[p]$ ，以最左端元素 $a[p]$ 为基准/划分元素，但排序结果为非递增序列，划分过程与书、讲义内容有差异】

void QuickSort (Type a[], int p, int r) (2 分)

```
{
    if (p < r) {
```

(3) (4 分) 在最好情况下，基准元素恰好选为排序元素的中位数，划分过程将待排序数组分为长度相等（或相差为 1）的 2 个子序列，(2 分)
此时 n 个元素排序的时间复杂度 $T(n)$ 满足：(2 分)

$$T(n) = \begin{cases} O(1), & n \leq 1 \\ 2T(n/2) + O(n), & n > 1 \end{cases}$$

经求解得 $T(n) = O(n \lg n)$

线性时间选择伪代码

```

Type Select(Type a[], int p, int r, int k)
{
    if (r-p<75) {
        用某个简单排序算法对数组a[p:r]排序;
        return a[p+k-1];
    };
    for ( int i = 0; i <= (r-p-4)/5; i++ ) {
        将a[p+5*i] 至 a[p+5*i+4]的第3小元素
        与a[p+i]交换位置;
    }
    //找中位数的中位数， r-p-4即上面所说的n-5
    Type x = Select(a, p, p+(r-p-4)/5, (r-p-4)/10);
    int i=Partition(a, p, r, x),
    j=i-p+1;
    if (k<=j) return Select(a, p, i, k);
    else return Select(a, i+1, r, k-j);
}

```

申大学

复杂度分析

$$T(n) \leq \begin{cases} C_1 & n < 75 \\ C_2 n + T(n/5) + T(3n/4) & n \geq 75 \end{cases}$$

$$T(n) = O(n)$$

上述算法将每一组的大小定为5，并选取75作为是否作递归调用的分界点。这2点保证了 $T(n)$ 的递归式中2个自变量之和 $n/5+3n/4=19n/20=\epsilon n$ ， $0<\epsilon<1$ 。这是使 $T(n)=O(n)$ 的关键之处。当然，除了5和75之外，还有其他选择。



最接近点对算法的复杂性分析

```
double cpair2(S)
```

```
{
```

```
    n=|S|;
```

```
    if (n < 2) return false;
```

```
1、 m=S中各点x间坐标的中位数;
```

```
    构造S1和S2;
```

```
    //S1={p∈S|x(p)≤m},
```

```
    //S2={p∈S|x(p)>m}
```

```
2、 d1=cpair2(S1);
```

```
    d2=cpair2(S2);
```

```
3、 dm=min(d1,d2);
```

4、 设P1是S1中距垂直分割线l的距离在dm之内的所有点组成的集合;

P2是S2中距分割线l的距离在dm之内所有点组成的集合;

将P1和P2中点依其y坐标值排序;

并设X和Y是相应的已排好序的点列;

5、 通过扫描X以及对于X中每个点检查Y中与其距离在dm之内的所有点(最多6个)可以完成合并;

当X中的扫描指针逐次向上移动时, Y中的扫描指针可在宽为2dm的区间内移动;

设dl是按这种扫描方式找到的点对间的最小距离;

```
6、 d=min(dm,dl);
```

```
    return d;
```

```
}
```

第3步和第6步是常数时间。

```
#define MAXFLOAT 3.14e38f;

struct Point_X
{
    float x;
    float y;
    int id;
    operator float()const{ return x; }
};

struct Point_Y
{
    float x;
    float y;
    int p;
    operator float()const { return y; }
};

/
template<class Type>
void Merge(Type *sd, Type *si, int left, int m, int right)
{
    int i = left, j = m + 1;
    int k = left;
    while (i <= m && j <= right)
    {
        sd[k++] = si[i] < si[j] ? si[i++] : si[j++];
    }
    while (i <= m)
    {
        sd[k++] = si[i++];
    }
}
```

```

    }
    while (j <= right)
    {
        sd[k++] = si[j++];
    }
}

template<class Type>
void Copy(Type *sd, Type *si, int left, int right)
{
    for (int i = left; i <= right; ++i)
    {
        sd[i] = si[i];
    }
}

template<class Type>
void MergePass(Type *br, Type *ar, int left, int right)
{
    if (left < right)
    {
        int m = (right - left) / 2 + left;
        MergePass(br, ar, left, m);
        MergePass(br, ar, m + 1, right);
        Merge(br, ar, left, m, right);
        Copy(ar, br, left, right);
    }
}

template<class Type>
void MergeSort(Type *ar, int n)
{
    Type *br = new Type[n];
    MergePass(br, ar, 0, n - 1);
    delete[] br;
}

///

void PrintPoint(Point_X *point, int num)
{
    for (int i = 0; i < NUM; i++)
    {
        cout << point[i].id << "=" << " " << point[i].x << setw(6) << point[i].y << endl;
    }
    cout << endl;
}

template<typename type>
float Distance(const type &p1, const type &p2)
{
    float dx = p1.x - p2.x;
    float dy = p1.y - p2.y;
    return sqrt(dx*dx + dy*dy);
}

float Closest(Point_X*X, Point_Y* Y, Point_Y*Z, int left, int right, Point_X&a,
Point_X&b)
{

```



```

int num = right - left;
if (num <= 0)
    return MAXFLOAT;
if (num == 1)
{
    a = X[left];
    b = X[right];
    return Distance(X[left], X[right]);
}
if (num == 2)
{
    float dis1=Distance(X[left], X[left+1]);
    float dis2 = Distance(X[left + 1], X[left + 2]);
    float dis3 = Distance(X[left + 2], X[left]);
    if (dis1 < dis2 && dis1 < dis3)
    {
        a = X[left];
        b = X[left+1];
        return dis1;
    }
    if (dis2 < dis1 && dis2 < dis3)
    {
        a = X[left+1];
        b = X[left+2];
        return dis2;
    }
    else
    {
        a = X[left];
        b = X[left+2];
        return dis3;
    }
}
int m = (right - left) / 2 + left;
//int z1 = left, m1 = m + 1;

float d1 = Closest(X, Y,Z, left, m, a, b);
Point_X a1, b1;
float d2 = Closest(X, Y, Z, m + 1, right, a1, b1);
float d = MAXFLOAT;
if (d1 < d2)
    d = d1;
else
{
    d = d2;
    a = a1;
    b = b1;
}
//Merge(Y, Z, left, m, right);
int k = left;
for (int i = left; i < right;i++)
{
    if (fabs(X[m].x - Y[i].x) < d)
    {

```

```

        Z[k++] = Y[i];
    }
}
for (int i = left; i < k; i++)
{
    for (int j = i + 1; j < k; j++)
    {
        if (fabs(Z[i].y - Z[j].y) >= d)//Z中本来就是按照Y排序的，因此一旦遇到比d大的
        的后面也就都比d大
            break;
        float d3 = Distance(Z[i], Z[j]);
        if (d3 > d)
        {
            a = X[Z[i].p];
            b = X[Z[j].p];
            d = d3;
        }
    }
}
return d;
}
float Cpair(Point_X*X, int num, Point_X &a, Point_X &b)
{
    MergeSort(X, num);
    Point_Y *Y = new Point_Y[num];
    for (int i = 0; i < num; i++)
    {
        Y[i].p = i;
        Y[i].x = X[i].x;
        Y[i].y = X[i].y;
    }
    Point_Y *Z = new Point_Y[num];
    MergeSort(Y, num);
    float mindis = Closest(X, Y, Z, 0, num-1, a, b);
    return mindis;
}

```

最接近点对问题

复杂度分析

$$T(n) = \begin{cases} O(1) & n < 4 \\ 2T(n/2) + O(n) & n \geq 4 \end{cases}$$

$$T(n) = O(n \log n)$$

第三章(1根据最优子结构得递归表达式 2代码 3复杂度)

矩阵连乘

2. 建立递归关系

- 设计算 $A[i:j]$, $1 \leq i \leq j \leq n$, 所需要的最少乘次数 $m[i,j]$, 则原问题的最优值为 $m[1,n]$
- 当 $i=j$ 时, $A[i:j]=A_i$, 因此, $m[i,i]=0$, $i=1,2,\dots,n$
- 当 $i < j$ 时, $m[i,j] = m[i,k] + m[k+1,j] + p_{i-1}p_kp_j$

这里 A_i 的维数为 $p_{i-1} \times p_i$

- 由最优子结构性质, 可以递归地定义 $m[i,j]$ 为:

$$m[i,j] = \begin{cases} 0 & i = j \\ \min_{i \leq k < j} \{m[i,k] + m[k+1,j] + p_{i-1}p_kp_j\} & i < j \end{cases}$$

k 的位置只有 $j-i$ 种可能

用动态规划法求最优解

```
void MatrixChain(int *p, int n, int **m, int **s)
```

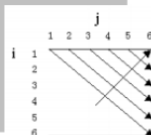
```
{
    for (int i = 1; i <= n; i++) m[i][i] = 0;
    for (int r = 2; r <= n; r++)
        for (int i = 1; i <= n - r + 1; i++) {
            int j = i + r - 1;
            m[i][j] = m[i+1][j] + p[i-1]*p[i]*p[j];
            s[i][j] = i;
            for (int k = i+1; k < j; k++) {
                int t = m[i][k] + m[k+1][j] + p[i-1]*p[k]*p[j];
                if (t < m[i][j]) { m[i][j] = t; s[i][j] = k; }
            }
        }
}
```

A1	A2	A3	A4	A5	A6
30×35	35×15	15×5	5×10	10×20	20×25

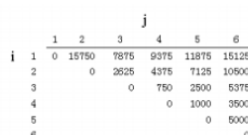
算法复杂度分析:

算法matrixChain的主要计算量取决于算法中对 r , i 和 k 的3重循环。循环体内的计算量为 $O(1)$, 而3重循环的总次数为 $O(n^3)$ 。因此, 算法的计算时间上界为 $O(n^3)$ 。

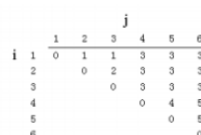
算法所占用的空间显然为 $O(n^2)$ 。



(a) 计算次序



(b) $m[i][j]$



(c) $s[i][j]$

```

void traceback (int i, int j, int ** s)
{
    if (i == j) return;
    traceback(s, i, s[i][j]);
    traceback(s, s[i][j] + 1, j);
    cout<<"Multiply A "<<i << ", " << s[i][j];
    cout<<" and A " << (s[i][j] + 1) << ", " << j << endl;
}

```

		j					
		1	2	3	4	5	6
i	1	0	15750	7875	9375	11875	15125
	2		0	2625	4375	7125	10500
	3			0	750	2500	5375
	4				0	1000	3500
	5					0	5000
	6						0

(b) $m[i][j]$

		j					
		1	2	3	4	5	6
i	1	0	1	1	3	3	3
	2		0	2	3	3	3
	3			0	3	3	3
	4				0	4	5
	5					0	5
	6						0

(c) $s[i][j]$

要输出A[1:n]的最优计算次序只要调用**traceback(1, n, s)**即可。对于上面所举的例子，通过调用**traceback(1, 6, s)**，可输出最优计算次序
 $((A_1(A_2A_3))((A_4A_5)A_6))$

最长公共子序列

(1) 根据最优子结构性质，建立子问题最优值的递归关系。

$c[i][j]$: 序列 $X(i)$ 和 $Y(j)$ 的最长公共子序列的长度, $X(i) = \{x_1, x_2, \dots, x_i\}$;
 $Y(j) = \{y_1, y_2, \dots, y_j\}$ 。

当 $i=0$ 或 $j=0$ 时，即其中一个序列为空，则空序列是 X 和 Y 的最长公共子序列，故此时 $c[i][j]=0$ 。

其它情况下，由最优子结构性质可建立递归关系如下：

$$c[i][j] = \begin{cases} 0 & i = 0, \text{ 或 } j = 0 \\ c[i-1][j-1] + 1 & i, j > 0; x_i = y_j \\ \max\{c[i][j-1], c[i-1][j]\} & i, j > 0; x_i \neq y_j \end{cases}$$

(2) 长度：

```

void LCSLength(int m, int n, char *x, char *y, int **c, int **b)
{

```

```

    int i, j;

```

```

    for (i = 1; i <= m; i++) c[i][0] = 0; /* 初始化, Y[j] 为空时

```

```

    for (i = 1; i <= n; i++) c[0][i] = 0; /* 初始化, X[i] 为空时

```



```

for (i = 1; i <= m; i++)
    Y(i)}
/*两重循环，自下而上，
   计算子问题 {X(i),
   for (j = 1; j <= n; j++) {
       if (x[i]==y[j]) {
           c[i][j]=c[i-1][j-1]+1;
           b[i][j]=1;}
       else if (c[i-1][j]>=c[i][j-1]) {
           c[i][j]=c[i-1][j]; b[i][j]=2;}
       else { c[i][j]=c[i][j-1];
           b[i][j]=3; }
   }
}

```

最长公共子序列

```

void LCS(int i, int j, char *x, int **b)
{

```

```

    if (i == 0 || j == 0) return;
    if (b[i][j] == 1) { LCS(i-1, j-1, x, b); cout<<x[i]; }
    /*第1种情况下，X(i)和Y(i)的最长公共子序列由X(i-1)和
    Y(i-1)的解 LCS(i-1, j-1, x, b)，加上位于最后的X[i]
    组成

```

```

    else if (b[i][j] == 2) LCS(i-1, j, x, b);
    else LCS(i, j-1, x, b); /*其它2种情况下，原问题解
    等于子问题

```

```

    解
}

```

(3) 算法时间复杂度

算法耗时: $O(mn)$

(4)

利用最长公共子序列求解最长递减子序列

由于最长递减子序列问题是求解一个给定序列的最长递减子序列，因此要用最长公共子序列问题来求解最长递减子序列问题，首先要构造出第二个序列，即让原序列和哪个序列求最长公共子序列才能得到原序列的最长递减子序列。显然，只需将原序列降

序排序，得到降序序列，再让原序列和降序序列求一下最长公共子序列，则这个最长公共子序列即为原序列的一个最长递减子序列。要注意的是，在最长公共子序列算法中，两个参数数组都是字符数组，在求解最长递减子序列问题时，需先将两个参数数组改为整型数组。算法如下：

```
void LIS (int a[], int x[], int m)
```

```
//该算法求解 x[1: m]这个长度为 m 的整型数组的最长递减子序列长度
```

```
{
```

```
for (int i=1;i<=m;i++) //输入数组 x
```

```
    scanf ("%d", &x[i]);
```

```
    for (int i=1;i<=m;i++) //数组 a 为数组 x 的副本
```

```
        a[i]=x[i];
```

```
    mergesort (a, 1, m); //对数组 a 进行递减归并排序
```

```
    LCSLength (x, a, m, m); //求数组 x 与数组 a 的最长公共子序列长度
```

```
}
```

最大子段和

记 $b[j] = \max_{1 \leq i \leq j} \{\sum_{k=i}^j a[k]\}$ ， $1 \leq j \leq n$ ，则所求的最大子段和为

$$\max_{1 \leq i \leq j \leq n} \sum_{k=i}^j a[k] = \max_{1 \leq j \leq n} \max_{1 \leq i \leq j} \sum_{k=i}^j a[k] = \max_{1 \leq j \leq n} b[j]$$

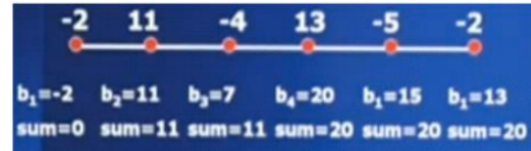
当 $b[j-1] > 0$ 时 $b[j] = b[j-1] + a[j]$ ，否则 $b[j] = a[j]$ 。由此可得计算 $b[j]$ 的动态规划递归式

$$b[j] = \max\{b[j-1] + a[j], a[j]\}, \quad 1 \leq j \leq n$$

```

int maxSum(int n, int * a)
{
    int sum=0, b=0;
    for (int i=1;i<=n;i++) {
        if (b>0) b+=a[i];
        else b=a[i];
        if (b>sum) sum=b;
    }
    return sum;
}

```



算法显然需要 $O(n)$ 计算时间

0/1背包问题

设所给0-1背包问题的子问题

$$\max \sum_{k=i}^n v_k x_k \quad \left\{ \begin{array}{l} \sum_{k=i}^n w_k x_k \leq j \\ x_k \in \{0,1\}, i \leq k \leq n \end{array} \right.$$

的最优值为 $m(i, j)$, 即 $m(i, j)$ 是背包容量为 j , 可选择物品为 $i, i+1, \dots, n$ 时0-1背包问题的最优值。由0-1背包问题的最优子结构性质, 可以建立计算 $m(i, j)$ 的递归式如下。

$$m(i, j) = \begin{cases} \max\{m(i+1, j), m(i+1, j-w_i) + v_i\} & j \geq w_i \\ m(i+1, j) & 0 \leq j < w_i \end{cases}$$

$$m(n, j) = \begin{cases} v_n & j \geq w_n \\ 0 & 0 \leq j < w_n \end{cases}$$

算法复杂度分析:

从 $m(i, j)$ 的递归式容易看出, 算法需要 $O(nc)$ 计算时间。当背包容量 c 很大时, 算法需要的计算时间较多。例如, 当 $c > 2^n$ 时, 算法需要 $\Omega(n2^n)$ 计算时间。

```

#include <deque>
#include <algorithm>
#include <iostream>
#include <fstream>
#include <stdio.h>
using namespace std;
#define rep(i, a, b) for (int i = a; i <= b; ++i)
class object{
public:
    int size;
    int value;
};

```



```

object thing[2500];
void find(int maxSize, int maxValue, int totalNum, int *qn, int q[][100][2]) {
    fstream fout("bag.out", ios::out);
    for (int i = 2; i <= totalNum+1; i++) {
        for (int j = 0; j < qn[i]; j++) {
            if (q[i][j][0] == maxSize && q[i][j][1] == maxValue) {
                fout << i-1 << " ";
                maxSize = maxSize - thing[i - 1].size;
                maxValue = maxValue - thing[i - 1].value;}
        }
    }
}

void change(int totalNum, int totalSize, int *qn, int *pn, int q[][100][2], int p[
[100][2]) {
    int maxValue = 0;
    int maxSize = 0;
    int temp;
    int x;
    for (int a = totalNum - 1; a > 0; a--) {
        for (x = 0, qn[a + 1] = 0; x < pn[a + 1]; x++) {
            if (p[a + 1][x][0] != -1) {
                temp = p[a + 1][x][0] + thing[a].size;
                if (temp <= totalSize) {
                    q[a + 1][qn[a + 1]][0] = temp;
                    q[a + 1][qn[a + 1]][1] = p[a + 1][x][1] + thing[a].value; qn[a
+ 1]++;
                }
            }
        }
        for (int i = 0; i < qn[i + 1]; i++) {
            for (int j = 0; j < pn[i + 1]; j++) {
                if (q[a + 1][i][0] >= p[a + 1][j][0] && q[a + 1][i][1] < p[a + 1]
[j][1]) {
                    q[a + 1][i][0] = -1;
                    q[a + 1][i][1] = -1;
                } else if (q[a + 1][i][0] <= p[a + 1][j][0] && q[a + 1][i][1] >
p[a + 1][j][1]) {
                    p[a + 1][j][0] = -1;
                    p[a + 1][j][1] = -1;
                }
            }
        }
        //???????p
        for (int i = 0; i < pn[a + 1]; i++) {
            if (p[a + 1][i][0] != 0) {
                p[a][pn[a]][0] = p[a + 1][i][0];
                p[a][pn[a]][1] = p[a + 1][i][1];
                pn[a]++;
            }
        }
        for (int i = 0; i < qn[a + 1]; i++) {
            if (q[a + 1][i][0] != -1) {
                p[a][pn[a]][0] = q[a + 1][i][0];
                p[a][pn[a]][1] = q[a + 1][i][1];
                pn[a]++;
            }
        }
    }
}

```

```

        }
    }
}
for (int i = 0; i < pn[1]; i++) {
    if (p[1][i][1] > maxValue) {
        maxValue = p[1][i][1];
        maxSize = p[1][i][0];
    }
}
find(maxSize, maxValue, totalNum, qn, q);
}
void start(int totalNum, int q[][100][2], int p[][100][2]) {
    q[totalNum + 1][0][0] = thing[totalNum].size;
    q[totalNum + 1][0][1] = thing[totalNum].value;
    p[totalNum][0][0] = 0;
    p[totalNum][0][1] = 0;
    p[totalNum][1][0] = thing[totalNum].size;
    p[totalNum][1][1] = thing[totalNum].value;
}
int main() {
    int totalSize;
    int totalNum;
    int pn[100] = { 0 }; //
    int qn[100] = { 0 }; //
    int p[100][100][2] = { -1 };
    int q[100][100][2] = { -1 };
    fstream fin("bag.in", ios::in);
    fin >> totalSize;
    fin >> totalNum;
    rep(i, 1, totalNum) {
        fin >> thing[i].size;
        fin >> thing[i].value;
    }
    int **m = new int*[2500];
    for (int i = 0; i < 2500; i++) {
        m[i] = new int[2500];
    }
    for (int x = 1; x <= totalNum; x++) {
        for (int y = 1; y <= totalSize; y++) {
            if (y < thing[x].size)
                m[x][y] = m[x - 1][y];
            else
                m[x][y] = max(m[x - 1][y], m[x - 1][y - thing[x].size] +
thing[x].value);
        }
    }
    start(totalNum, q, p);
    pn[totalNum] = 2;
    qn[totalNum + 1] = 1;
    change(totalNum, totalSize, qn, pn, p, q);
}

```

算法改进

$$m(i, j) = \begin{cases} \max\{m(i+1, j), m(i+1, j-w_i) + v_i\} & j \geq w_i \\ m(i+1, j) & 0 \leq j < w_i \end{cases}$$

- 函数 $m(i, j)$ 是由函数 $m(i+1, j)$ 与函数 $m(i+1, j-w_i) + v_i$ 作 $\max$ 运算得到的。因此，函数 $m(i, j)$ 的全部跳跃点包含于函数 $m(i+1, j)$ 的跳跃点集 $p[i+1]$ 与函数 $m(i+1, j-w_i) + v_i$ 的跳跃点集 $q[i+1]$ 的并集中。
- 易知， $(s, t) \in q[i+1]$ 当且仅当 $w_i \leq s \leq c$ 且 $(s-w_i, t-v_i) \in p[i+1]$ 。
- 因此，容易由 $p[i+1]$ 确定跳跃点集 $q[i+1]$ 如下 $q[i+1] = p[i+1] \oplus (w_i, v_i) = \{(j+w_i, m(i, j) + v_i) | (j, m(i, j)) \in p[i+1]\}$
- 另一方面，设 (a, b) 和 (c, d) 是 $p[i+1] \cup q[i+1]$ 中的2个跳跃点，则当 $c \geq a$ 且 $d < b$ 时， (c, d) 受控于 (a, b) ，从而 (c, d) 不是 $p[i]$ 中的跳跃点。除受控跳跃点外， $p[i+1] \cup q[i+1]$ 中的其它跳跃点均为 $p[i]$ 中的跳跃点。
- 由此可见，在递归地由表 $p[i+1]$ 计算表 $p[i]$ 时，可先由 $p[i+1]$ 计算出 $q[i+1]$ ，然后合并表 $p[i+1]$ 和表 $q[i+1]$ ，并清除其中的受控跳跃点得到表 $p[i]$ 。

上述算法的主要计算量在于计算跳跃点集 $p[i] (1 \leq i \leq n)$ 。由于 $q[i+1] = p[i+1] \oplus (w_i, v_i)$ ，故计算 $q[i+1]$ 需要 $O(|p[i+1]|)$ 计算时间。合并 $p[i+1]$ 和 $q[i+1]$ 并清除受控跳跃点也需要 $O(|p[i+1]|)$ 计算时间。从跳跃点集 $p[i]$ 的定义可以看出， $p[i]$ 中的跳跃点相应于 $x_1, \dots, x_n$ 的0/1赋值。因此， $p[i]$ 中跳跃点个数不超过 2^{n-i+1} 。由此可见，算法计算跳跃点集 $p[i]$ 所花费的计算时间为 $O\left(\sum_{i=2}^n |p[i+1]|\right) = O\left(\sum_{i=2}^n 2^{n-i}\right) = O(2^n)$ 。从而，改进后算法的计算时间复杂性为 $O(2^n)$ 。当所给物品的重量 $w_i (1 \leq i \leq n)$ 是整数时， $|p[i]| \leq c+1, (1 \leq i \leq n)$ 。在这种情况下，改进后算法的计算时间复杂性为 $O(\min\{nc, 2^n\})$ 。

```
include <deque>
#include <algorithm>
#include <iostream>
#include <fstream>
using namespace std;
#define rep(i, a, b) for (int i = a; i <= b; ++i)
class object{
public:
    int size;
    int value;
};
object thing[2500];
void find(int i, int j, int* record, int **m)
{
```

```

    if (i > 0)
    {
        if (m[i][j] == m[i - 1][j])
        {
            record[i - 1] = 0;
            find(i - 1, j,
                record,m);
        }
        else if (j - thing[i].size >= 0 && m[i][j] == m[i - 1][j - thing[i].size]
+ thing[i].value)
        {
            record[i - 1] = 1;
            find(i - 1, j - thing[i].size,
                record,m);
        }
    }
}

int main() {
    int totalSize;
    int totalNum;
    fstream fin("bag.in", ios::in), fout("bag.out", ios::out);
    fin >> totalSize;
    fin>>totalNum;
    rep(i, 1, totalNum) {
        fin>>thing[i].size;
        fin>>thing[i].value;
    }
    int **m = new int*[2500];
    for (int i = 0; i < 2500; i++){
        m[i] = new int[2500];
    }

    int x ;
    int y;
    for (x = 1; x <= totalNum; x++) {
        for (y = 1; y <= totalSize; y++) {
            if (y < thing[x].size)
                m[x][y] = m[x - 1][y];
            else
                m[x][y] = max(m[x - 1][y], m[x - 1][y - thing[x].size] +
thing[x].value);
        }
    }
    int* record;
    record = (int*)malloc(sizeof(int) * totalNum);
    find(totalNum, totalSize, record,m);

    for (int i = 0; i < totalNum; i++)
    {
        if(record[i]==1)
            fout << i+1 << " ";
    }
    return 0;
}

```

第四章(准确性,代码,复杂度)

活动安排(找活动结束晚的)

首先,我们按照按照活动结束时间非递增排序,即从小到大得到一个序列 $E\{1, 2, \dots, n\}$,我们可以知道1这个活动是一定会被选到的,因为他的结束时间最早,我们可以有更多的时间去观看其他比赛。

假设具有一个最优解A,且A中的活动安排也是按照活动的结束时间非递增排序,即从小到大排序的,且他的第一个活动为K,这时候我们就需要重点比较K和1的关系了:

如果 $K = 1$: 我们就可以知道A就是满足以贪心选择开始的一个最优解,所以证明的结束了;这个就满足了第一个要求;

如果 $K \neq 1$: 我们就需要设一个解为B,这个B不包含A中的K这个活动,但是他包含A中的其他活动,且他的第一个活动为E中的1,我们可以想象,这个是求解最多能够观看多少场比赛的问题,所以,由E的排序可知,活动1的结束时间比活动K的结束时间还要早,所以我们可以知道B也是一个最优解,且是一个由贪心选择开始的最优解,因为他们能观看的比赛场次都是相同的,都达到了最大值。得证第一个要求;

二、最优子结构的问题

在做了第一步贪心选择活动1之后,原问题简化为了,对E中所有与活动1相容的活动进行安排的子问题,用通俗的话说,就是现在1已经确定了,然后我们就需要对剩下的活动进行安排,但是这些安排有一个前提,就是他们的开始时间,大于等于1的结束时间。

所以,若A是原问题的最优解,则设 $A' = A - \{1\}$ 是接下来的子问题的最优解,也就是活动开始时间 > 1 活动的结束时间的剩余的活动,设为E',的最优解,这时候我们可以用反证法,假设E'有一个解B',B'比A'包含更多的活动,则 $B = B' + \{1\}$,则容易知道B比最优解A还要包含更多的活动,与A的最优性矛盾,所以不存在这样一个B',也就是得到

结论:每一次所做的贪心选择都将原问题简化为与原问题具有相同形式的子问题,得证,活动安排问题,具有最优子结构的性质。

```
void GreedySelector(int n, Type s[], Type f[], bool A[])
{
    A[1]=true;
    int j=1;
    for (int i=2;i<=n;i++) {
        if (s[i]>=f[j]) { A[i]=true; j=i; }
        else A[i]=false;
    }
}
```

各活动的起始时间和
结束时间存储于数组
s和f中且按结束时间
的非减序排列

由于输入的活动以其完成时间的**非减序**排列，所以算法**greedySelector**每次总是选择**具有最早完成时间**的相容活动加入集合A中。直观上，按这种方法选择相容活动为未安排活动留下尽可能多的时间。也就是说，该算法的贪心选择的意义是**使剩余的可安排时间段极大化**，以便安排尽可能多的相容活动。

算法**greedySelector**的效率极高。当输入的活动已按结束时间的非减序排列，算法只需 **$O(n)$** 的时间安排n个活动，使最多的活动能相容地使用公共资源。如果所给出的活动未按非减序排列，可以用 **$O(n\log n)$** 的时间重排。

最优装载

```
void Loading(int x[], Type w[], Type c, int n)
{
    int *t = new int [n+1];
    Sort(w, t, n);
    for (int i = 1; i <= n; i++) x[i] = 0;
    for (int i = 1; i <= n && w[t[i]] <= c; i++)
    {
        x[t[i]] = 1;
        c -= w[t[i]];
    }
}
```

算法**loading**的主要计算量在于将集装箱依其重量从小到大排序，故算法所需的计算时间为 **$O(n\log n)$** 。

贪心选择性质

最优装载问题具有贪心选择性质。

证明：设集装箱已按重量**从小到大**排序， $\{x_1, x_2, \dots, x_n\}$ 是最优装载问题的一个最优解。

设 k 为最先装入箱中的最轻货物，即

$$k = \min_{1 \leq i \leq n} \{i | x_i = 1\}$$

E. g. 最优解为 $\{1, 1, 0, 1, 1\}$ ，则 $k=1$ ；

最优解为 $\{0, 1, 0, 1, 1\}$ ，则 $k=2$ ；

当问题有最优解（有可能是非贪心策略解）时，

(1) 当 $k=1$ ，即最轻的第1个货物被装入，货物选择的放入顺序符合贪心策略， $\{x_1, x_2, \dots, x_n\}$ 是满足贪心选择性质的一个最优解

北京邮电大学

(2) 当 $k>1$ 时，最轻的货物1没有被装入。

e. g. 最优解为 $\{0, 1, 0, 1, 1\}$ ，则 $k=2$ ；

按如下方式从 $\{x_1, x_2, \dots, x_n\}$ 构造出一个满足贪心策略的最优解 $\{y_1, y_2, \dots, y_k, \dots, y_n\}$ ， $y_1=0$ ：

i) 对 k ，其中 $1 < k \leq n$ ， $y_k=1$ ，取 $y_1=1$ ， $y_k=0$ ，将第1个货物放入，第 k 个货物拿出，即用第1个货物替换原方案中最轻的货物，而第1个货物比第 k 个货物更轻，即 $w_1 \leq w_k$ 。

ii) $y_i=x_i$ ， $1 \leq i \leq n$ ， $i \neq k$ ，其它货物是否被放入仍保持不变

e. g. 原最优解 $\{x_1, x_2, \dots, x_n\} = \{0, 1, 0, 1, 1\}$ ， $k=2$ 。

改为 $\{y_1, y_2, \dots, y_n\} = \{1, 0, 0, 1, 1\}$ ， $k=1$

新方案仍然满足容量约束条件：

$$\sum_{i=1}^n w_i y_i = w_1 - w_k + \sum_{i=1}^n w_i x_i \leq \sum_{i=1}^n w_i x_i \leq c$$

北京邮电大学

因此，新方案 Y 是一个满足贪心策略的可行解。

需进一步判断装入的集装箱数是否最多？

$$\text{又由于 } \sum_{i=1}^n y_i = \sum_{i=1}^n x_i$$

说明2个方案 Y 具有与最优方案一样的装箱数，所以：优先将最轻的第1个货物装入可以使得装载的总箱数最大化!!!

综上所述， Y 是一个满足贪心策略的最优解。

北京邮电大学

最优装载问题具有最优子结构性质。

证明：

假设货物 $\{1, 2, \dots, n\}$ 已按重量从小到大排序。

对原问题：待装货物为 $\{1, 2, \dots, n\}$ 、容量为 C ， $\{x_1, x_2, \dots, x_n\}$ 是其满足贪心策略的1个最优解，且 $x_1=1$ 。

e.g. $\{x_1, x_2, \dots, x_n\} = \{1, 0, 1, 0, 1\}$ ， $k=1$ 。

则对子问题：待装货物为 $\{2, \dots, n\}$ 、容量为 $C-w_1$ ， $\{x_2, \dots, x_n\}$ 是一个最优解，e.g. $\{x_2, \dots, x_n\} = \{0, 1, 0, 1\}$ ，

由最优装载问题的贪心选择性质和最优子结构性质，容易证明算法**loading**的正确性。

哈夫曼编码

```
#include <stdio.h>
#include <string.h>

typedef struct {
    int weight;          // 结点权值?
    int parent, lc, rc;  // 双亲结点和左 右子节点
} HTNode, *HuffmanTree;

void Select(HuffmanTree &HT, int n, int &s1, int &s2)
{
    int minum;          // 定义一个临时变量保存最小值?
    for(int i=1; i<=n; i++)    // 以下是找到第一个最小值
    {
        if(HT[i].parent == 0)
        {
            minum = i;
            break;
        }
    }
    for(int i=1; i<=n; i++)
    {
        if(HT[i].parent == 0)
            if(HT[i].weight < HT[minum].weight)
                minum = i;
    }
    s1 = minum;
    // 以下是找到第二个最小值，且与第一个不同
    for(int i=1; i<=n; i++)
    {
```

```

        if(HT[i].parent == 0 && i != s1)
        {
            minum = i;
            break;
        }
    }
    for(int i=1; i<=n; i++)
    {
        if(HT[i].parent == 0 && i != s1)
            if(HT[i].weight < HT[minum].weight)
                minum = i;
    }
    s2 = minum;
}

void CreatHuff(HuffmanTree &HT, int *w, int n)
{
    int m, s1, s2;
    m = n * 2 - 1; // 总结点的个数
    HT = new HTNode[m + 1]; // 分配空间
    for(int i=1; i<=n; i++) // 1 - n 存放叶子结点, 初始化
    {
        HT[i].weight = w[i];
        HT[i].parent = 0;
        HT[i].lc = 0;
        HT[i].rc = 0;
    }
    for(int i=n+1; i<=m; i++) // 非叶子结点的初始化
    {
        HT[i].weight = 0;
        HT[i].parent = 0;
        HT[i].lc = 0;
        HT[i].rc = 0;
    }

    printf("\nthe HuffmanTree is: \n");

    for(int i = n+1; i<=m; i++) // 创建非叶子节点, 建哈夫曼树
    { // 在HT[1]~HT[i-1]的范围内选择两个parent为0且weight最小的两个结点, 其序号分别赋值
      给 s1 s2
        Select(HT, i-1, s1, s2);
        HT[s1].parent = i; // 删除这两个结点
        HT[s2].parent = i;
        HT[i].lc = s1; // 生成新的树, 左右子节点是 s1和s2
        HT[i].rc = s2;
        HT[i].weight = HT[s1].weight + HT[s2].weight; // 新树的权??
        printf("%d (%d, %d)\n", HT[i].weight, HT[s1].weight, HT[s2].weight);
    }
    printf("\n");
}

int main()*
{
    HuffmanTree HT;

```

```
int *w, n, wei;
printf("input the number of node\n");
scanf("%d", &n);
w = new int[n+1];
printf("\ninput the %dth node of value\n", n);

for(int i=1; i<=n; i++)
{
    scanf("%d", &wei);
    w[i] = wei;
}
CreatHuff(HT, w, n);

return 0;
}
```

• 证明思路:

设字符集C的1个最优前缀码表示为二叉树T。

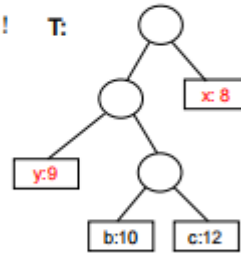
采用一定方法, 将T修改新树T', 使得

- 1) 在T'中具有最小频率的x和y是最深叶子, 且互为兄弟
 - 2) T'还是C的最优前缀。
- 这样x、y在最优前缀码T'中只有最后一位不同。

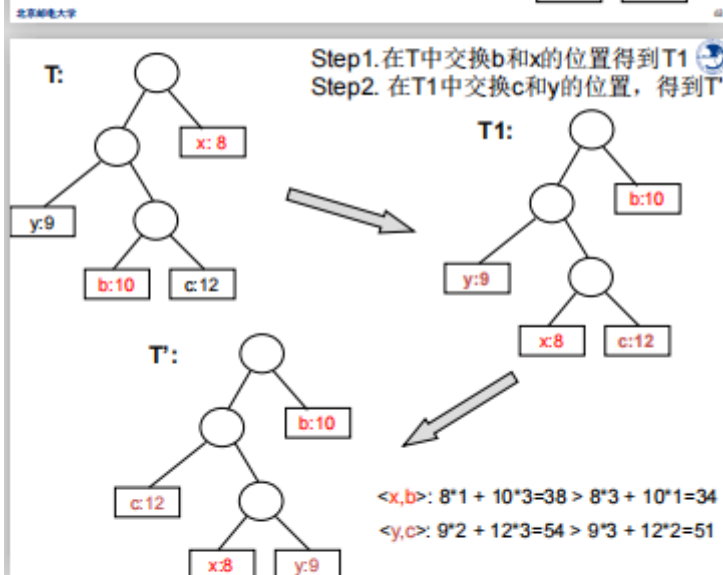
假设: b、c是T中最深叶子且互为兄弟, $f(b) \leq f(c)$;

已知: C中2个最小频率字符 $f(x) \leq f(y)$,

但在T中, x、y有可能并非最深结点!! T:



由于x、y具有最小频率,
故 $f(x) \leq f(b)$, $f(y) \leq f(c)$



• 可以证明:

- 1) $B(T) - B(T1) \leq 0$, 即第一步交换不会增加平均码长
- 2) $B(T1) - B(T') \leq 0$, 即第二步交换也不会增加平均码长

故T'的码长仍然是最短的, 即T'是最优前缀码, 并且其最小频率的x、y具有最深的深度(最长的编码), 且只有最后一位不同。

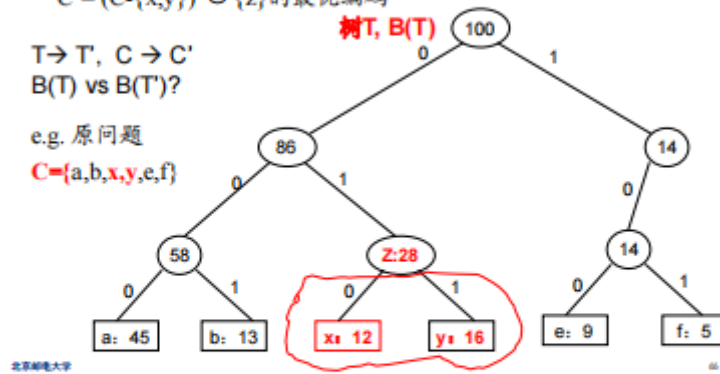
• 构造性证明:

对T中2个互为兄弟的叶节点x、y，z为其父节点。
将z看做频率为 $f(z)=f(x)+f(y)$ 的字符，则 $T'=T-\{x, y\}$ 是子问题
 $C'=(C-\{x, y\}) \cup \{z\}$ 的最优编码

$T \rightarrow T', C \rightarrow C'$
 $B(T) \text{ vs } B(T')$

e.g. 原问题

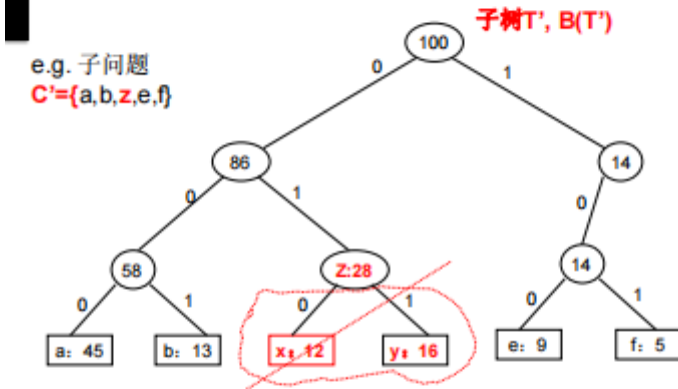
$C=\{a, b, x, y, e, f\}$



北京邮电大学

e.g. 子问题

$C'=\{a, b, z, e, f\}$



北京邮电大学

• 证明关键点:

1) T的平均码长 $B(T)$ 可用子树 T' 的平均码长 $B(T')$ 表示

$$B(T) = B(T') + 1 * f(x) + 1 * f(y)$$

上式: 递推表达式, 表示原问题最优值与子问题最优值之间的关系

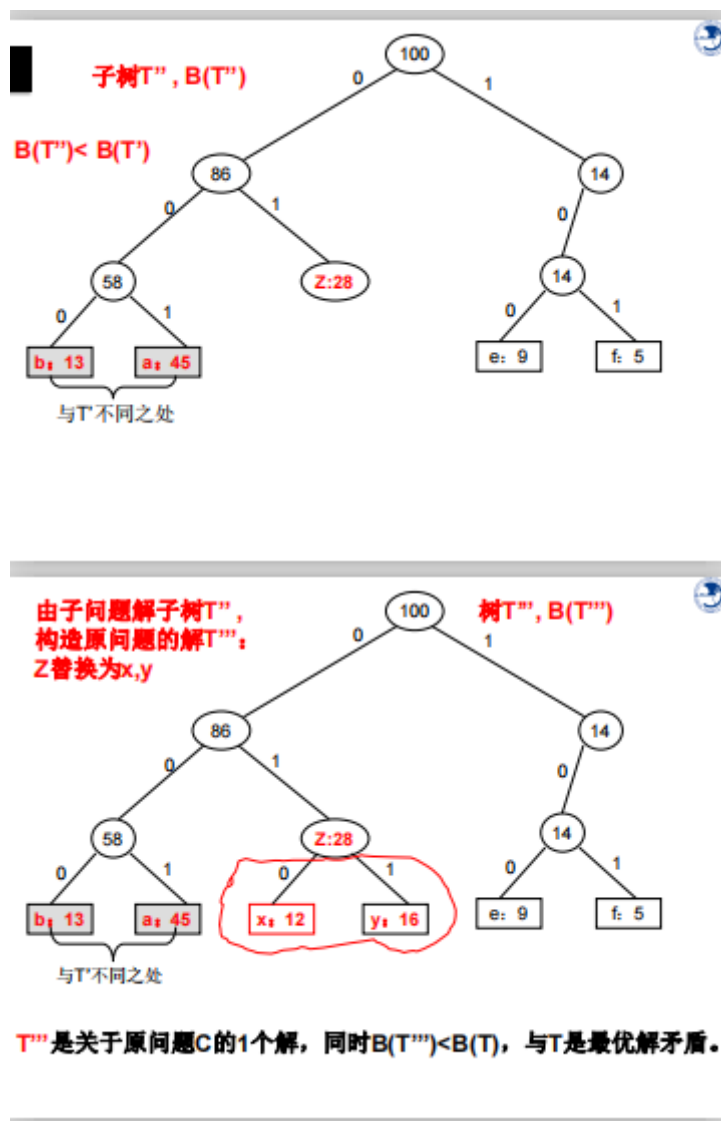
2) T' 所表示的 C' 的前缀码的码长 $B(T')$ 是最短/最优的
反证法证明:

假设有另一个 T'' , 是子问题 C' 的最优前缀码, 即
 $B(T') > B(T'')$.

节点z在 T'' 还是一个叶节点, 在 T'' 中将z替换为其子节点x、y, 得到 T''' .

e.g 见下页:

北京邮电大学



$$T = O(C \log C)$$

迪杰斯特拉

(2) 先证明贪心性质

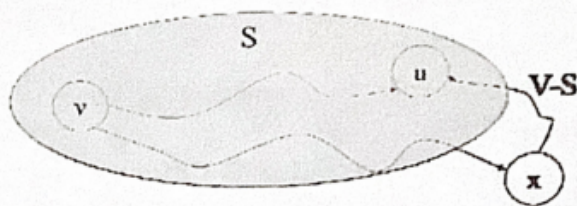
贪心选择策略:

在每步迭代时, 从 $V-S$ 中选择具有最短特殊路径 $\text{dist}[u]$ 的顶点 u , 加入 S

贪心策略正确性: 需证明 对任意顶点 u ,

从 v 开始、经过 G 中任意顶点到达 u 的全局最短路径的长度 $d(v, u)$

= 从 v 开始、只经过 S 中顶点到达 u 的最短路径的长度 $\text{dist}(u)$
 即: 不存在另一条 v 到 u 的最短路径, 该路径上某些节点 x 不在 $V-S$ 中,
 且该路径的长度 $d(v, u) < \text{dist}[u]$.



假设:

(1) 在迭代求解过程中, 顶点 u 是遇到的第 1 个满足: $d(v, u) < \text{dist}[u]$,
 即 $d(v, u) \neq \text{dist}[u]$, 的顶点

(2) 从 v 到 u 的全局最短路径上, 第 1 个属于 $V-S$ 的顶点为 x
 首先, 因为 u 是第一个满足全局最短路径不完全在 S 集合中的顶点, 即

$$d(v, u) < \text{dist}[u]$$

, 而 x 是在 u 之前遇到的顶点, x 的最短路径完全在 S 中, 因此:

$$\text{dist}[x] = d(v, x) \leq d(v, u)$$

对 v 到 u 的全局最短路径, 有

$$d(v, x) + \text{distance}(x, u) = d(v, u) < \text{dist}[u]$$

由于 $\text{distance}(x, u) > 0$, 因此

$$\text{dist}[x] = d(v, x) \leq d(v, u) < \text{dist}[u], \text{ 即}$$

$$\text{dist}[x] < \text{dist}[u]$$

但是根据路径 p 构造方法, 在下图所示情况下, u, x 都在集合 S 之外, 即 u, x 都属于 $V-S$, 但 u 被选中时, 并没有选 x , 根据扩展 S 的原则——选 dist 最小的顶点加入 S , 说明此时:

$$\text{dist}[u] \leq \text{dist}[x]$$

这与 前面推出的

$$\text{dist}[x] < \text{dist}[u]$$

相矛盾

从而证明了贪心性质

再证明最优子结构

对顶点 u , 考察将 u 加到 S 之前和之后, $\text{dist}[u]$ 的变化, 添加 u 之前的 S 称为老 S , 加入 u 之后的 S 称为新 S 。

要求: u 加到 S 中后, $\text{dist}[u]$ 不增加。

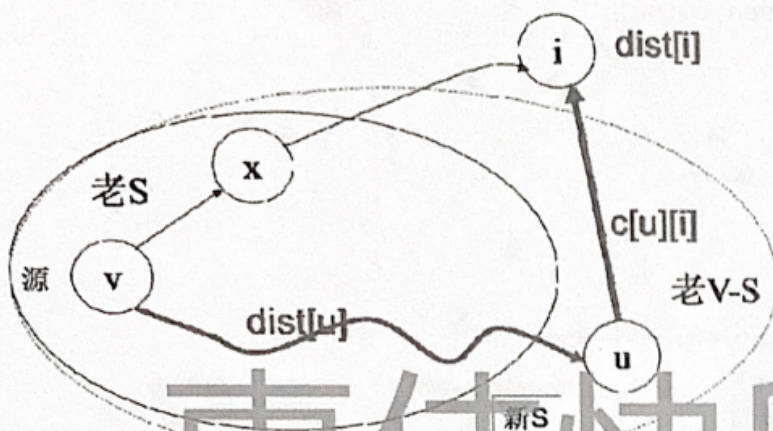
对另外 1 个节点 i , 考察 u 的加入对 $\text{dist}[i]$ 的影响:

1) 假设添加 u 后, 出现 1 条从 v 到 i 的新路, 该路径先由 v 经过老 S 中

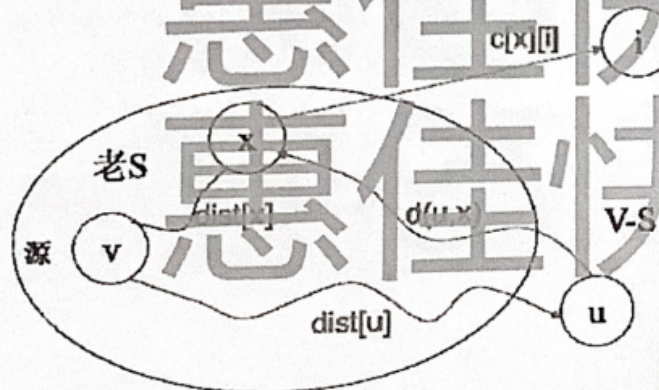
的顶点到达 u ，再从 u 经过一条直接边到达 i

该路径的最短长度 = $\text{dist}[u] + c[u][i]$

如果 $\text{dist}[u] + c[u][i] < \text{原来的 dist}[i]$ ，则算法用 $\text{dist}[u] + c[u][i]$ 替代 $\text{dist}[i]$ ，得到新的 $\text{dist}[i]$ 。否则， $\text{dist}[i]$ 不更新。



2) 如果新路径如下图所示，先经过 u ，再回到 S 中的 x ，由 x 直接到达 i 。
 x 处于老的 S 中，故 $\text{dist}[x]$ 已经是最短路径， x 比 u 先加入 S ，因此
 $\text{dist}[x] \leq \text{dist}[u] + d(u, x)$



此时，从原 v 到 i 的最短路径 $\text{dist}[i]$ 小于路径 (v, u, x, i) 的长度，因此算法更新 $\text{dist}[i]$ 时不需要考虑该路径， u 的加入对 $\text{dist}[i]$ 无影响。因此，无论算法中 $\text{dist}[u]$ 的值是否变化，它总是关于当前顶点集合 S 的到顶点 u 的最短路径。

也就是说：对于加入 u 之前、之后的新老 S 所对应的 2 个子问题，算法执行过程保证了 $\text{dist}[u]$ 始终是 u 的最优解



计算复杂性

用带权邻接矩阵表示具有 n 个顶点和 e 条边的带权有向图 $G(V, E)$.

Dijkstra算法的主循环体需要 $O(n)$ 时间。这个循环需要执行 $n-1$ 次，所以完成循环需要 $O(n^2)$ 时间。

算法的其余部分所需要时间不超过 $O(n^2)$

prime



Prim算法

设 $G=(V, E)$ 是连通带权图， $V=\{1,2,\dots,n\}$ 。

Prim算法的**基本原理**：

Step1. 首先置顶点集合 $S=\{1\}$

Step2. 当 S 是 V 的真子集时，作如下的**贪心选择**：

选取满足条件 $i \in S, j \in V-S$ ，且 $c[i][j]$ 最小的边 $\langle i, j \rangle$ ，将顶点 j 添加到 S 中，边 $\langle i, j \rangle$ 加到边集 T 中。

Step3. 重复上述过程，直到 $S=V$ 为止，此时边集 T 就是最小生成树

在这个过程中选取到的所有边恰好构成 G 的一棵**最小生成树**。

算法复杂性： $O(n^2)$



Prim算法正确性说明

利用最小生成树性质和数学归纳法（对顶点集合 S 归纳）容易证明，上述算法中的**边集合 T 始终包含 G 的某棵最小生成树中的边**：

贪心选择性质、最优子结构性质

因此，在算法结束时， T 中的所有边构成 G 的一棵最小生成树。

```
const int MAXN = 1000, INF = 0x3f3f3f3f; // 定义一个INF表示无穷大。
int g[MAXN][MAXN], dist[MAXN], n, m, res;
// 我们用g[][]数组存储这个图，dist[]储存到集合S的距离，res保存结果。
bool book[MAXN]; // 用book数组记录某个点是否加入到集合S中
int main()
```

```

{
    cin>>n>>m;//读入这个图的点数n和边数m
    for(int i = 1 ; i<= n ;i++)
    {
        for(int j = 1; j <= n ;j++)
        {
            g[i][j] = INF;//初始化任意两个点之间的距离为正无穷（表示这两个点之间没有边）
        }
        dist[i] = INF;//初始化所有点到集合S的距离都是正无穷
    }
    for(int i = 1; i <= m ; i++)
    {
        int a,b,w;
        cin>>a>>b>>w;//读入a, b两个点之间的边
        g[a][b] = g[b][a] = w;//由于是无向边，我们对g[a][b]和g[b][a]都要赋值
    }
    prim();//调用prim函数
    if(res==INF)//如果res的值是正无穷，表示不能该图不能转化成一棵树，输出orz
        cout<<"orz";
    else
        cout<<res;//否则就输出结果res
    return 0;
}

void prim()
{
    dist[1] = 0;//把点1加入集合S，点1在集合S中，将它到集合的距离初始化为0
    book[1] = true;//表示点1已经加入到了S集合中
    for(int i = 2 ; i <= n ;i++)dist[i] = min(dist[i],g[1][i]);//用点1去更新dist[]
    for(int i = 2 ; i <= n ; i++)
    {
        int temp = INF;//初始化距离
        int t = -1;//接下来去寻找离集合S最近的点加入到集合中，用t记录这个点的下标。
        for(int j = 2 ; j <= n; j++)
        {
            if(!book[j]&&dist[j]<temp)//如果这个点没有加入集合S，而且这个点到集合的距离小于temp就将下标赋给t
            {
                temp = dist[j];//更新集合V到集合S的最小值
                t = j;//把点赋给t
            }
        }
        if(t==-1){res = INF ; return ;}
        //如果t==-1，意味着在集合V找不到边连向集合S，生成树构建失败，将res赋值正无穷表示构建失败，结束函数
        book[t] = true;//如果找到了这个点，就把它加入集合S
        res+=dist[t];//加上这个点到集合S的距离
        for(int j = 2 ; j <= n ; j++)dist[j] = min(dist[j],g[t][j]);//用新加入的点更新dist[]
    }
}
}

```

Kruskal算法时间复杂度

当图的边数为 e 时，Kruskal算法所需的计算时间

$$O(e \log e)$$

与Prim算法比较

1. 当 $e = \Omega(n^2)$ 时，Kruskal算法比Prim算法差，
2. 当 $e = o(n^2)$ 时，Kruskal算法比Prim算法好得多

证明：由Kruskal算法构成的任何生成树 $T^* = G[\{e_1, e_2, \dots, e_{n-1}\}]$ 都是最小生成树，这里 n 为赋权图 G 的顶点数。（个人理解：因为局部最优，取尽可能小的权，不代表全局最小，因此正确性还是要证明的吧）

使用反证法，1、有kruskal算法构成的生成树 T^* 和异于 T^* 的生成树 T ，这两种生成树。

2、定义函数 $f(T)$ 表示不在 T 中的最小权值的边 e_i 。假设 T^* 不是最小树， T 真正的小树，显然 T 会使 $f(T)$ 尽可能大的，即 T 本身权重则会尽可能小，。

3、设 $f(T)=k$,表示存在一个不在 T 中的最小权值边 $e_k=k$ ，也就是说 $e_1, e_2, \dots, e_{k-1}$ 同时在 T 和 T^* 中， $e_k=k$ 不在 T 中

4、 $T+e_k$ 包含唯一圈 C 。设 $e_{k'}$ 是 C 的一条边，他在 T 中而不在 T^* 中。（想象圈 C 中至少有 e_k 和 $e_{k'}$ ，其中 e_k 是又Kruskal算法得出的最小权边）

5、令 $T' = W(T) + w(e_i) - w(e_{i'})$ ，kruskal算法选出的是最小权边 e_k ，（而 $e_{k'}$ 是 T 自己根据 $f(T)$ 选出来的边）有 $w(e_{k'}) \geq w(e_k)$ 且 $W(T') = W(T^*)$ （ T' 也是一个最小生成树）

6、但是 $f(T') > k = f(T)$ ，即 T 没有做到使得 $f(T)$ 尽可能大，不再是真正的小树，所以 $T=T^*$,从而 T^* 确实是一棵最小树。

```
int father[N]; //并查集数组
int findFather(int x) //并查集查询函数
{
    ...
}
//kruskal函数返回最小生成树的边权之和，参数n为顶点个数，m为图的边数
int kruskal(int n, int m)
{
    //ans为所求边权之和，Num_Edge为当前生成树的边数
    int ans = 0, Num_Edge = 0;
    for(int i = 1; i <= n; i++) //假设题目中顶点范围是[1,n]
    {
        father[i] = i; //并查集初始化
    }
    sort(E, E + m, cmp); //所有边按边权从小到大排序
```



```

for(int i = 0; i < m; i++) //枚举所有边
{
    int faU = findFather(E[i].u); //查询测试边两个端点所在集合的根节点
    int faV = findFather(E[i].v);
    if(faU != faV) //如果不在一个集合中
    {
        father[faU] = faV; //合并集合(即把测试边加入最小生成树)
        ans += E[i].cost; //边权之和增加测试边的边权
        Num_Edge++; //当前生成树的边数加1
        if(Num_Edge == n - 1)
            break; //边数等于顶点数减1时结束算法
    }
}
if(Num_Edge != n - 1)
    return -1; //无法连通时返回-1
else
    return ans; //返回最小生成树的边权之和
}

```

第五章(解向量,枝剪策略解空间树,代码.时间复杂度)

装载问题

在类Loading中增加两个私有数据成员x和bestx。

- » X记录从根至当前结点的路径,x[i]=0/1;[1 0 1 1 0]
- » Bestx记录当前最优解,算法搜索到达一个叶子结点处,就修正bestx的值。

```

class loading
{
    friend Type MaxLoading(Type [], Type int);
    //初始化操作
private:
    void Backtrack( int i); //对解空间搜索
    int n; //集装箱数
    *x, //当前解, 数组
    *bestx //当前最优解, 数组
    Type *w, //集装箱重量数组
    c, //第一艘轮船的载重量
    cw, //当前载重量
    bestw; //当前最优载重量
    r; //剩余集装箱的重量
};

```

```

template < class Type >
void Loading< Type > :: Backtrack( int i )
{
    if (i > n)
    {
        if (cw > bestw)
        {
            for(j =1; j<=n;j++)
                bestx[j] =x[j];
            bestw = cw;
        }
        return;
    }
    //搜索子树
    r -= w[i];
    if ( cw + w[i] <= c)
    { //x[i] = 1; //二叉树的左孩子
        x[i] =1;
        cw+= w[i];
        Backtrack(i+1);
        cw -= w[i]; //回溯
    }
    if( cw+r > bestw)
    {
        x[i] = 0;
        Backtrack(i+1); //x[i] =0; //二叉树的右孩子
    }
    r += w[i];
}

```

复杂度分析

由于bestx可能被更新 $O(2^n)$ 次，改进后算法的计算时间复杂性为 $O(n2^n)$ 。

```

template < class Type>
Type MaxLoading (Type w[], Type c, int n)
{ //返回最优载重量
    Loading<Type> X;
    //初始化x
    X.w = w;
    X.c = c; //第一艘
    X.n = n;
    X.bestw = 0;
    X.cw = 0;
    X.r = 0;
    for( int i=1; i<=n; i++)
        X.r += w[i];
    //计算最优载重量
    X.Backtrack(1);
    return X.bestw;
}

```

背包

回溯法的过程中具有深度优先遍历过程

考虑遍历的结点数量，考虑结点个数即可

最多 $2^n - 1$ 个结点

时间复杂度就是遍历 $2^n - 1$ 个结点： $O(2^n)$

解空间树是子集树时，时间复杂度通常是 $O(2^n)$

解空间树是排序树时，时间复杂度通常是 $O(n!)$



通用0/1背包 – 回溯法求解

```
Typep Knap<Typew, Typep>::Bound(int i)
{ // 计算上界
    Typew cleft = c - cw; // 剩余容量
    Typep b = cp;
    // 以物品单位重量价值递减序装入物品
    while (i <= n && w[i] <= cleft) {
        cleft -= w[i];
        b += p[i];
        i++;
    }
    // 装满背包
    if (i <= n) b += p[i]/w[i] * cleft;
    return b;
}
```

•解空间：子集树

•可行性约束函数：

$$\sum_{i=1}^n w_i x_i \leq c$$

•上界函数：

计算右子树中解的上界的更好方法是将剩余物品依其单位重量价值排序，然后依次装入物品，直至装不下时，再装入该物品的一部分而装满背包。由此得到的价值是右子树中解的上界。

通用0/1背包

```
template <class Typew, class Typep>
void Knap < Typew, Typep> :: Backtrack(int i)
{
    if (i > n) //到达叶子节点
    {
        bestp = cp;
        return;
    }
    if ((cw + w[i] <= c)
    {
        cw += w[i];
        cp += p[i];
        Backtrack(i + 1);
        cw -= w[i];
        cp -= p[i];
    }
    if (Bound ( i + 1) > bestp)
        Backtrack( i + 1 );
}
```

- **问题描述：**设有 n 个作业 $\{J_1, J_2, \dots, J_n\}$ 需要处理，每个作业 J_i ($1 \leq i \leq n$) 都有两项任务组成。两项任务需要分别在 2 台机器即机器 1 和机器 2 上处理。要求每个作业 J_i 的第一项任务在机器 1 上处理，第二项任务在机器 2 上处理，并且第一项任务在机器 1 上处理完后，第二项任务才能在机器 2 上开始处理。规定每个作业 J_i 用 $f(1, i)$ 记录其在机器 1 上的处理时间（该时间是指从机器 1 启动到该作业完成的时间）。每个作业 J_i 用 $f(2, i)$ 记录其在机器 2 上的处理时间（该时间是指从机器 2 启动到该作业完成的时间）。不同的作业调度方案处理完成所有作业所需的时间显然不同。批处理作业调度要求制定最佳作业调度方案，使其完成的时间和最小：

$$F = \min \sum_{i=1}^n f_{2,i}$$

例如：有 3 个作业 $\{J_1, J_2, J_3\}$ 需要处理，作业 J_i 在机器 1 和机器 2 上的处理时间如下图。找出最优调度方案。

作业 J_i 在机器 1 和机器 2 上的处理时间

	机器 1	机器 2
作业 1	2	1
作业 2	3	1
作业 3	2	3

- **分析：**

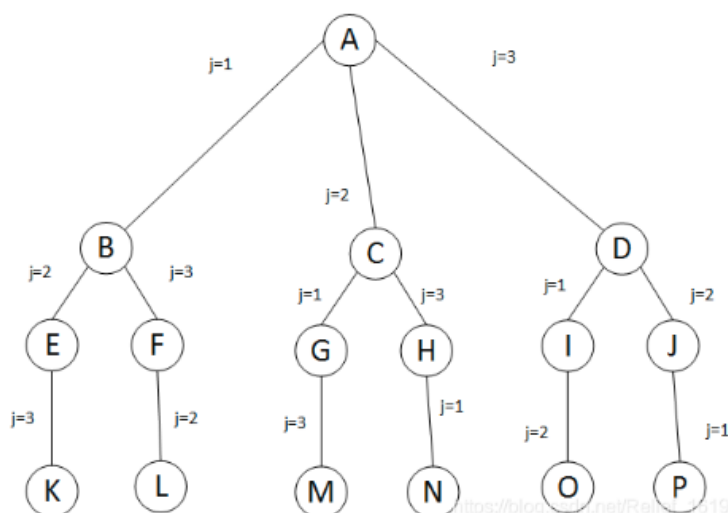
1、定义解空间

因为 n 个作业有 $n!$ 种调度方案，因此，批处理作业调度问题的解空间对应 n 个作业的全排列。3 个作业的调度方案对应 3 个作业的全排列。3 个作业的全排列有 6 种，即 1,2,3、1,3,2、2,1,3、2,3,1、3,2,1、3,1,2。该问题的解决要求是，需要确定一种作业顺序，使得从第一个作业在机器 1 上开始处理到最后一个作业在机器 2 完成所需处理的时间最短。

2、建立批处理解空间结构

该问题的解空间可以表示为一颗排列树。其解空间树有 $n!$ 个叶结点，解空间树从树的根结点到叶结点的路径定义了批处理作业调度问题的一个调度方案。

如图：



3、采用回溯法以深度优先的策略搜索解空间

在搜索过程中，当 $i > n$ 时，表示搜索到达一个叶结点，得到一个新的调度方案，该方案由解空间树从树的根结点到叶结点的路径定义。

如果 $i \leq n$ ，则当前的可扩展结点是解空间树中的一个内部结点。此时，选择下一个要安排的作业，以深度优先的方式递归地对相应的子树进行搜索。

为了节省搜索成本，对于每个内部结点，在进入子树前，需要用限界函数检查其是否满足限界条件。若满足限界条件，则有可能产生新的作业调度方案，子结点为可行结点，搜索向其下一层进行；若不满足限界条件，则减去以其为根结点的子树，搜索往回移动，移至离其最近的活结点处，然后以相同的策略在解空间树中递归地进行搜索。

4、时间复杂性分析

由于算法 backtrack 在每个结点处耗费 $O(1)$ 计算时间，所以在最坏情况下，批处理作业调度问题的回溯算法的总时间耗费是 $O(n!)$ 。

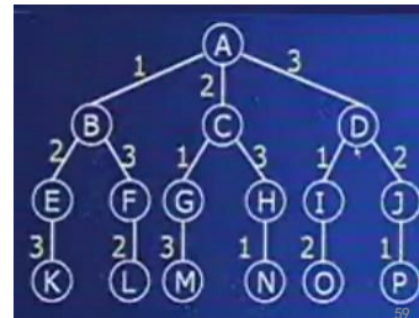
批处理作业调度问题

```
void Flowshop::Backtrack(int i)
```

```
{
    if (i > n) {
        for (int j = 1; j <= n; j++)
            bestx[j] = x[j];
        bestf = f;
    }
    else
        for (int j = i; j <= n; j++) {
            f1+=M[x[j]][1];
            f2[i]=((f2[i-1]>f1)?f2[i-1]:f1)+M[x[j]][2];
            f+=f2[i];
            if (f < bestf) {
                Swap(x[i], x[j]);
                Backtrack(i+1);
                Swap(x[i], x[j]);
            }
            f1-=M[x[j]][1];
            f-=f2[i];
        }
}
```

北京邮电大学

t_{ji}	机器1	机器2
作业1	2	1
作业2	3	1
作业3	2	3



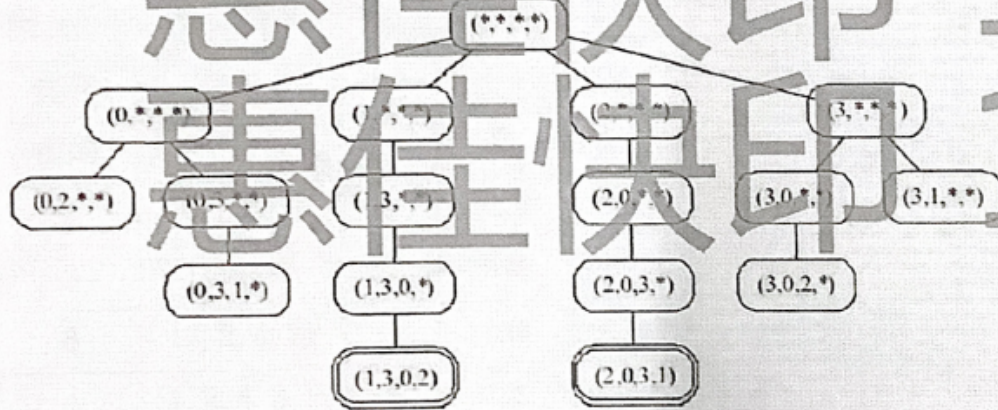
N皇后

答案如下：

(1) 给棋盘上的行和列从 1 到 4 编号，同时也给皇后从 1 到 4 编号。由于每一个皇后应放在不同的行上，不失一般性，假设皇后 i 放在第 i 行上，因此 4 皇后问题可以表示成 4 元组 $(x_1, x_2, \dots, x_4)$ ，其中 $x_i (i=1, 2, \dots, 4)$ 表示皇后 i 所放置的列号。

(2) 约束条件 1: 当 $i \neq j$ 时, $x_i \neq x_j$ (式 1)

约束条件 2: $|i - x_i| \neq |j - x_j|$ (式 2)



(3) 代码

```
#include<iostream>
#include<math.h>
using namespace std;
int n=8;
int total=0;
```

```

int *c=new int(n);
bool is_ok(int row){
    for(int j=0;j!=row;j++){
        if(c[row]==c[j] || row-c[row]==j-c[j] ||
row+c[row]==j+c[j])
            return false;
    }
    return true;
}
void queen(int row){
    if(row==n)
        total++;
    else
        for(int col=0;col!=n;col++){
            c[row]=col;
            if(is_ok(row))
                queen(row+1);
        }
}
int main(){
    queen(0);
    cout<<total;
    return 1;
}

```

(4) 时间复杂度

时间复杂度为: $O(N!)$

分析：使用一个数组记录每行放置的皇后的列下标，依次在每一行放置一个皇后。每次新放置的皇后都不能和已经放置的皇后之间有攻击：即新放置的皇后不能和任何一个已经放置的皇后在同一列以及同一条斜线上，并更新数组中的当前行的皇后列下标。当 N 个皇后都放置完毕，则找到一个可能的解。当找到一个可能的解之后，将数组转换成表示棋盘状态的列表，并将该棋盘状态的列表加入返回列表。

由于每个皇后必须位于不同列，因此已经放置的皇后所在的列不能放置别的皇后。第一个皇后有 N 列可以选择，第二个皇后最多有 $N-1$ 列可以选择，第三个皇后最多有 $N-2$ 列可以选择（如果考虑到不能在同一条斜线上，可能的选择数量更少），因此所有可能的情况不会超过 $N!$ 种，遍历这些情况的时间复杂度是 $O(N!)$ 。

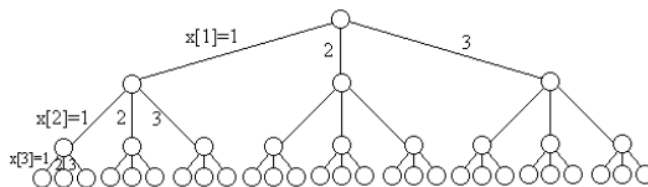
图着色

图的m着色问题

- 解向量: $(x_1, x_2, \dots, x_n)$ 表示顶点 i 所着颜色 $x[i]$
- 可行性约束函数: 顶点 i 与已着色的相邻顶点颜色不重复。

```
void Color::Backtrack(int t)
{
    if (t > n) {
        sum++;
        for (int i = 1; i <= n; i++)
            cout << x[i] << ' ';
        cout << endl;
    }
    else
        for (int i = 1; i <= m; i++) {
            x[t] = i;
            if (Ok(t)) Backtrack(t + 1);
        }
}

bool Color::Ok(int k)
{
    // 检查颜色可用性
    for (int j = 1; j <= n; j++)
        if ((a[k][j] == 1) && (x[j] == x[k])) return false;
    return true;
}
```



复杂度分析

图 m 可着色问题的解空间树中内结点个数是 $\sum_{i=0}^{n-1} m^i$
 对于每一个内结点, 在最坏情况下, 用 ok 检查当前扩展结点的每一个儿子所相应的颜色可用性需耗时 $O(mn)$ 。因此, 回溯法总的时间耗费是

$$\sum_{i=0}^{n-1} m^i (mn) = nm(m^n - 1) / (m - 1) = O(nm^n)$$

TSP问题的回溯求解

- 旅行回路总费用极小化

$$\min \left\{ \sum_{i=1}^{n-1} w(v_i, v_{i+1}) + w(v_n, v_1) \right\}$$

式中，权 $w(v_i, v_j)$ （或： $w(i, j)$ ）表示城市 i 与 j 间的直接距离， $w(v_i, v_j) = \infty$ 表示城市 i 与 j 间无直接路径。

将图中 n 个顶点编号为 $1, 2, \dots, n$;

以**顶点1**为起点，旅行回路描述为： $1, x_1, x_2, \dots, x_n, 1$;

其中， $x_1, x_2, \dots, x_n$ 为顶点 $2, 3, 4, \dots, n$ 的一个排列。

因此，解空间大小为 $(n-1)!$

剪枝条件/约束1：如果当前正在考虑的顶点 j 与当前路径中的末端结点 i 没有边相连，即 $w[i, j] = \infty$ ，则不必搜索 j 所在分支

E.g. 当前已有的部分路径为 $\langle 1, 2, ?, ? \rangle$ ，路径末端结点为2;

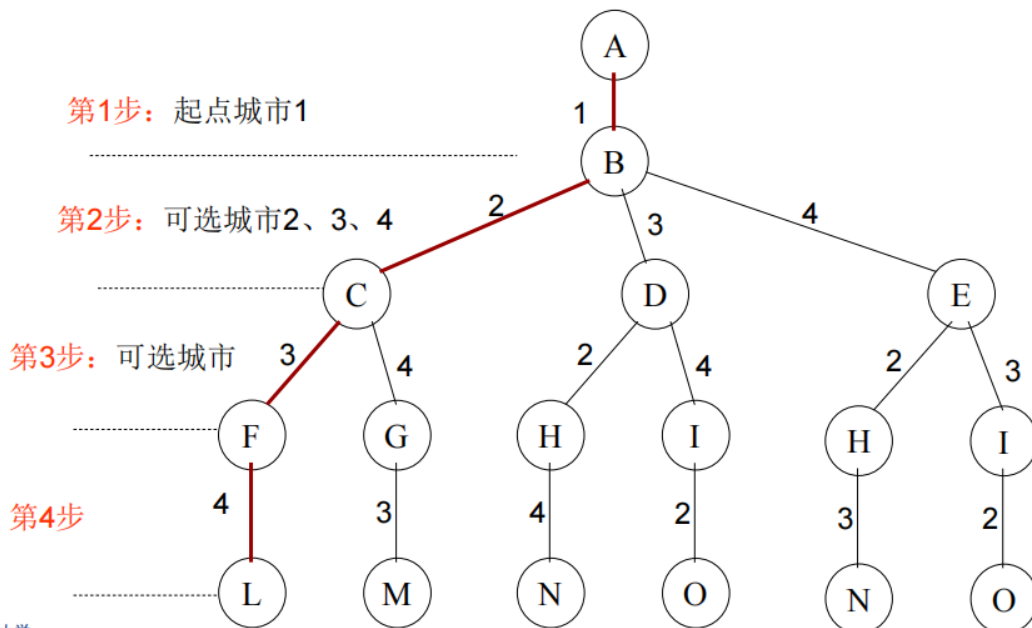
由此得到**剪枝/约束条件2**：x[n]

如果 $B(i) \geq \text{bestw}$ ，则停止搜索 $x[i]$ 分支及其下面的层，

其中， bestw 代表到目前为止，在前面的搜索中，从其它已经搜索过的路径中，找到的最佳完整回路的权和（总长度）

TSP问题的回溯求解

求解过程：以深度优先的方式，从树根结点开始，依次扩展树结点，直到达到叶结点——搜索过程中动态产生解空间



北京邮电大学

82

$bestx[1:n]$: 记录最佳路径; cw : 当前部分路径的总长



BacktrackTSP(i)

1. if $i=n$ then //已经搜索到叶节点，已经选择了最后1个城市
 - {
2. if $w(x[n-1], x[n]) \neq \infty$ and $w(x[n], 1) \neq \infty$
 - // 最后1个城市与前一城市相连、与第1个城市相连，组成1个满足条件的回路
3. then //比较当前回路与以前最佳回路，看当前回路是否更优
4. if $cw + w(x[n-1], x[n]) + w(x[n], 1) < bestw$
5. then { //当前回路更优，更新最优搜索结果
6. $bestw = cw + w(x[n-1], x[n]) + w(x[n], 1)$;
7. for $j=1$ to n do $bestx[j]=x[j]$
- }
- }

北京邮电大学

91

```

1. if i=n then //已经搜索到叶节点, 已经选择了最后1个城市
    {
        ....
7.         for j=1 to n do bestx[j]=x[j]
    }
8. else //如果搜索没有到叶节点, 当前得到的部分路径
        <1, x[2], ..., x[i-1], ?, ..., ?>
    {
9.         for j=i to n do //考察x[i]的各个可能取值
10.            if w(x[i-1], x[j])  $\neq \infty$  and cw + w(x[i-1], x[j]) < bestw
                // 向当前部分路径加入新城市x[j]后, x[j]与x[i-1]相
                连, 且扩展后的部分路径<1, x[2], ..., x[i-1], x[j], ..., ?>
                的成本小于当前最优回路的长度
11.            then //继续扩展、搜索次路径

```

```

9.         for j=i to n do //考察x[i]的各个可能取值
10.            if w(x[i-1], x[j])  $\neq \infty$  and cw + w(x[i-1], x[j]) < bestw
                // 向当前部分路径加入新城市x[j]后, x[j]与x[i-1]相
                连, 且扩展后的部分路径<1, x[2], ..., x[i-1], x[j], ..., ?>
                的成本小于当前最优回路的长度
11.            then //继续扩展、搜索次路径
                {
12.                swap(x[i], x[j]) //加入第i个城市
13.                cw = cw + w(x[i-1], x[i]) //更新扩展后的路径的代价
14.                BacktrackTSP(i+1) //递归搜索以x[i]为根的
                                    后续子树
15.                cw = cw - w(x[i-1], x[i]) //搜索失败, 回溯, 回到
                                    第9步, 为搜索x[i]的另一个取值x[j+1]准备
16.                swap(x[i], x[j])
                }

```

backtrack最坏情况下, 需要每条路都走一遍, 即 $(n-1)!$ 种走法, 时间复杂度 $O(n!)$